

a. Core loop Use standard stick-fighter combat (strike, dash, jump) and a physics-based grappling hook to defeat a gauntlet of AI bosses. Every victory forces your grappling hook to inherit a broken physics bug from the dead boss, while you simultaneously dodge the ghost of that boss replaying its final attacks in the next round.

b. The three systems

1. The Tether & Combat Engine (Owner: Jeremiah)
 - State it owns: Hitboxes, base movement (jump/dash velocities), and Tether physics (anchor coordinates, rope tension, elasticity).
 - What it does: Handles all the standard stick-figure fighting and Godot physics. The tether acts as the player's primary combo-extender and high-speed mobility tool, allowing them to swing off walls to build momentum.
2. The Corrupt Inheritance System (Owner: Daryl)
 - State it owns: The Boss Queue, the Active Glitch ID, and the tether modifier floats.
 - What it does: When a boss dies, this system extracts their defining "bug" and forces it onto the player's grappling hook for the next round. Example glitches include "Slippery Anchor" (grapple slides down the wall) or "Rubber Band" (tether elasticity is heavily multiplied).
3. The Phantom Echo System (Owner: Caleb)
 - State it owns: An array of the previous boss's last 10 seconds of inputs/positions, the Phantom's active hitbox, and playback speed.
 - What it does: Spawns a ghostly stick-figure of the boss you just killed. The ghost runs on a loop, executing the exact attacks the boss used before it died, acting as an unpredictable environmental hazard while you fight the current boss.

c. The seams

- Seam 1: **apply_corruption**
 - What crosses: Tether physics modifier values (e.g., the friction float of the anchor point).
 - Who writes: Corrupt Inheritance System (Daryl).
 - Who reads: Tether & Combat Engine (Jeremiah) overwrites its base grappling hook variables.
 - *Freeze Test:* If you freeze the tether's **anchor_friction** variable at 0, the grappling hook will instantly fail to hold the player's weight, sliding completely off the ceiling the moment they try to swing, proving it dictates physical behavior and not just visuals.
- Seam 2: **target_prediction**
 - What crosses: The player's current **velocity** Vector2 (including both their dash speed and tether swing speed).

- Who writes: Tether & Combat Engine (Jeremiah).
- Who reads: Phantom Echo System (Caleb). Instead of blindly replaying its recorded attacks exactly where they originally happened, the Phantom reads the player's current velocity and shifts its spawn coordinates to intercept the player mid-swing or mid-dash.
- *Freeze Test*: If you freeze the player's **velocity** Vector2 at 0, the Phantom will only ever spawn directly on top of the player's current stationary position, completely ignoring their momentum.

d. Your reading of Inheritance Inheritance is literal legacy code and lingering grudges. It is a load-bearing mechanic because it attacks the player on two fronts: physically and spatially. You inherit a codebase you cannot rewrite (the dead boss's glitch), forcing a mandatory physical mutation that ruins your grappling hook's reliability. Simultaneously, you inherit their ghost, which forces you to pay the cost for how you fought in the previous round.

e. Art direction

- **Style/Vibe**: High-contrast 90s cyberpunk anime (stark Black, White, Cyan, and Magenta). Fast, kinetic, and gritty.
- **Character Designs**: Highly readable, stylized 2D stick figures with glowing outlines to distinguish the Player, the Current Boss, and the Phantom.
- **Animation**: Minimal animation frames. The game relies entirely on Godot's physics-based movement, heavy hit-stop, extreme screen shake, and speed lines to sell the impact of collisions.
- **UI**: Minimalist. Standard chunky arcade health bars and a simple round timer.

f. Scope

- **What is in**: Basic jump/dash movement, the grappling hook physics engine, 1 static arena, 3 boss AI profiles, and the ghost playback system.
- **Cut 1 (The Fourth System)**: An upgrade or shop system between rounds.
- **Cut 2**: Complex sprite rigging. We are using static stick-figure poses and rotating them dynamically based on the grapple hook's tension angle.
- **Cut 3**: Projectile weapons. We are sticking exclusively to melee strikes and the grappling hook to keep the combat scope contained and the mechanics focused on spatial positioning.

g. Which seam breaks first, and why **apply_corruption** (Seam 1) will break first. Daryl's system is injecting variable overrides directly into Jeremiah's Godot tether math. If the Inheritance system passes an unexpected value—like multiplying the grapple's elasticity by a negative number, or setting the max rope length to zero—Godot's joint physics will likely bug out and instantly catapult the player's stick figure completely out of the arena's collision bounds.

